Youssef Hesham Mahmoud SE3

Lab 04

1.1 product-service/app.py:

app.py E:\lab4\product-service\app.py

```python
from flask import Flask, jsonify

app = Flask(__name__)

PRODUCTS = {
    1: {"id": 1, "name": "Laptop", "price": 1200},
    2: {"id": 2, "name": "Phone", "price": 650},
    3: {"id": 3, "name": "Headphones", "price": 120}
}

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"service": "product-service", "status": "up"}), 200

@app.route("/products/<int:product_id>", methods=["GET"])
def get_product(product_id):
    product = PRODUCTS.get(product_id)
    if product:
        return jsonify(product), 200
    return jsonify({"error": "Product not found"}), 404

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5001)
```

1.2 order-service/app.py:

app.py E:\lab4\order-service\app.py

```python
import os
import time
import requests
from flask import Flask, jsonify, request
app = Flask(__name__)
PRODUCT_SERVICE_URL = os.getenv("PRODUCT_SERVICE_URL", "http://product-service:5001")

def fetch_product(product_id, retries=2, delay=1):
    url = f"{PRODUCT_SERVICE_URL}/products/{product_id}"
    for attempt in range(retries + 1):
        try:
            response = requests.get(url, timeout=2)
            return response
        except requests.exceptions.RequestException:
            if attempt < retries:
                time.sleep(delay)
            else:
                return None
@app.route("/health", methods=["GET"])
def health():
    return jsonify({"service": "order-service", "status": "up"}), 200
@app.route("/orders", methods=["POST"])
def create_order():
    data = request.get_json()
    product_id = data.get("product_id")
    quantity = data.get("quantity", 1)
    response = fetch_product(product_id)
    if response is None:
        return jsonify({"error": "product-service unavailable"}), 50
    if response.status_code != 200:
        return jsonify({"error": "invalid product"}), 40
    product = response.json()
    total = product["price"] * quantity
    return jsonify({
        "message": "Order created",
        "product": product["name"],
        "quantity": quantity,
```


```python
    product = response.json()
    total = product["price"] * quantity
    return jsonify({
        "message": "Order created",
        "product": product["name"],
        "quantity": quantity,
```


```python
        "total_price": total
    }), 201
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5002)
```

2.1 product-service/Dockerfile

Dockerfile E:\lab4\product-service\Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5001
CMD ["python", "app.py"]
```

2.2 order-service/Dockerfile

Dockerfile E:\lab4\order-service\Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5002
CMD ["python", "app.py"]
```

docker-compose.yml

docker-compose.yml E:\lab4\docker-compose.yml

```yaml
services:
  product-service:
    build: ./product-service
    container_name: product-service
    ports:
      - "5001:5001"
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:5001/health')"]
      interval: 10s
      timeout: 5s
      retries: 3
    restart: unless-stopped

  order-service:
    build: ./order-service
    container_name: order-service
    ports:
      - "5002:5002"
    environment:
      PRODUCT_SERVICE_URL: http://product-service:5001
    depends_on:
      - product-service
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:5002/health')"]
      interval: 10s
      timeout: 5s
      retries: 3
    restart: unless-stopped
```

Screenshots of Successful API Calls

```
PS E:\lab4> docker compose ps
NAME                IMAGE                    COMMAND            SERVICE
 CREATED            STATUS                      PORTS
order-service       lab4-order-service       "python app.py"    order-service
 19 minutes ago     Up 19 minutes (healthy)    0.0.0.0:5002->5002/tcp, [::]:500
2->5002/tcp
product-service     lab4-product-service     "python app.py"    product-service
 19 minutes ago     Up 19 minutes (healthy)    0.0.0.0:5001->5001/tcp, [::]:500
1->5001/tcp
PS E:\lab4>
```

Activate Windows
Go to Settings to activate Windows.

Screenshot 2 Product service health:

```
PS E:\lab4> Invoke-WebRequest -Uri "http://localhost:5001/health" -UseBasicP
arsing


StatusCode        : 200
StatusDescription : OK
Content           : {"service":"product-service","status":"up"}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 44
                    Content-Type: application/json
                    Date: Sun, 26 Apr 2026 07:36:59 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"service":"product-service","stat...
Forms             :
Headers           : {[Connection, close], [Content-Length, 44],
                    [Content-Type, application/json], [Date, Sun, 26 Apr
                    2026 07:36:59 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        :
RawContentLength  : 44
```

Screenshot 3 — Get existing product:

```
PS E:\lab4> Invoke-WebRequest -Uri "http://localhost:5001/products/1" -UseBa
sicParsing


StatusCode        : 200
StatusDescription : OK
Content           : {"id":1,"name":"Laptop","price":1200}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 38
                    Content-Type: application/json
                    Date: Sun, 26 Apr 2026 07:37:58 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"id":1,"name":"Laptop","price":12...
Forms             :
Headers           : {[Connection, close], [Content-Length, 38],
                    [Content-Type, application/json], [Date, Sun, 26 Apr
                    2026 07:37:58 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        :
RawContentLength  : 38
```

Screenshot 4 — Get non-existing product:

```
PS E:\lab4> Invoke-WebRequest -Uri "http://localhost:5001/products/99" -UseB
asicParsing
Invoke-WebRequest : {"error":"Product not found"}
At line:1 char:1
+ Invoke-WebRequest -Uri "http://localhost:5001/products/99" -UseBasicP ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest
   :HttpWebRequest) [Invoke-WebRequest], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.Powe
   rShell.Commands.InvokeWebRequestCommand
```

Screenshot 5 — Successful order:

```
PS E:\lab4> Invoke-WebRequest -Uri "http://localhost:5002/orders" -Method PO
ST -Headers @{"Content-Type"="application/json"} -Body '{"product_id": 1, "q
uantity": 2}' -UseBasicParsing


StatusCode        : 201
StatusDescription : CREATED
Content           : {"message":"Order created","product":"Laptop","quantity
                    ":2,"total_price":2400}

RawContent        : HTTP/1.1 201 CREATED
                    Connection: close
                    Content-Length: 79
                    Content-Type: application/json
                    Date: Sun, 26 Apr 2026 07:38:55 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"message":"Order created","p...
Forms             :
Headers           : {[Connection, close], [Content-Length, 79],
                    [Content-Type, application/json], [Date, Sun, 26 Apr
                    2026 07:38:55 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        :
RawContentLength  : 79
                                                   Activate Windows
```

Failure Simulation Screenshots Screenshot 6 — Stop product-service:

```
PS E:\lab4> docker stop product-service
product-service
PS E:\lab4>
```

Activate Windows
Go to Settings to activate

Screenshot 7 — Order fails with 503:

```
PS E:\lab4> Invoke-WebRequest -Uri "http://localhost:5002/orders" -Method PO
ST -Headers @{"Content-Type"="application/json"} -Body '{"product_id": 2, "q
uantity": 1}' -UseBasicParsing
Invoke-WebRequest : {"error":"product-service unavailable"}
At line:1 char:1
+ Invoke-WebRequest -Uri "http://localhost:5002/orders" -Method POST -H ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest
   :HttpWebRequest) [Invoke-WebRequest], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.Powe
   rShell.Commands.InvokeWebRequestCommand
```

Activate Windows
Go to Settings to activate Windows.

Screenshot 8 — Restart and recover:

```
PS E:\lab4> docker start product-service
product-service
PS E:\lab4> docker compose ps
NAME              IMAGE                  COMMAND           SERVICE
 CREATED          STATUS                            PORTS
order-service     lab4-order-service     "python app.py"   order-service
 27 minutes ago   Up 27 minutes (healthy)           0.0.0.0:5002->5002/tcp,
[::]:5002->5002/tcp
product-service   lab4-product-service   "python app.py"   product-service
 27 minutes ago   Up 2 seconds (health: starting)   0.0.0.0:5001->5001/tcp,
[::]:5001->5001/tcp
PS E:\lab4>
```

Activate Windows
Go to Settings to activate Windows.

Screenshot 9 — Order works again:

```
PS E:\lab4> Invoke-WebRequest -Uri "http://localhost:5002/orders" -Method PO
ST -Headers @{"Content-Type"="application/json"} -Body '{"product_id": 1, "q
uantity": 2}' -UseBasicParsing


StatusCode        : 201
StatusDescription : CREATED
Content           : {"message":"Order created","product":"Laptop","quantity
                    ":2,"total_price":2400}

RawContent        : HTTP/1.1 201 CREATED
                    Connection: close
                    Content-Length: 79
                    Content-Type: application/json
                    Date: Sun, 26 Apr 2026 07:44:40 GMT
                    Server: Werkzeug/3.1.8 Python/3.12.13

                    {"message":"Order created","p...
Forms             :
Headers           : {[Connection, close], [Content-Length, 79],
                    [Content-Type, application/json], [Date, Sun, 26 Apr
                    2026 07:44:40 GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        :
RawContentLength  : 79
```

Architecture reflection task:

Q1: Which parts show benefits of microservices over monolith?
Each service is deployed independently. product-service and order-service can be updated, scaled, or restarted without affecting each other. If one crashes, the other keeps running.
Q2: Which new complexities were introduced?
"Network communication between services can fail. We now need health checks, retries, and timeouts. Debugging is harder because errors can happen across service boundaries." **Q3: What would break if network latency increased?**
"The order-service has a 2-second timeout. If product-service becomes slow, orders will fail with 503 errors. High latency causes cascading failures in synchronous microservice designs." **Q4: Which 12-factor app principles are visible?**
"Config: PRODUCT_SERVICE_URL is set via environment variable. Processes: both services are stateless. Port binding: each service binds to its own port. Dependencies: requirements.txt declares all dependencies explicitly."